

SECOM Yield Optimization

Predicting low-yield wafers in semiconductor manufacturing

Industrial IoT · Smart-Factory Yield · Semiconductor Packaging & Test

1,567

wafers

590

sensors

~14:1

pass : fail

4

runnable notebooks

Why this problem — and why it matters

Semiconductor packaging & test is a high-value, high-demand domain, and central to Malaysia

The business question

Which process signals predict low-yield (fail) wafers, and how can the fab use them to raise yield and cut per-unit cost?

- Every wafer passes hundreds of monitored process steps.
- A failed wafer is lost yield and wasted cost.
- Features are anonymized, so we deliver ranked sensor IDs as engineering leads.

6th

largest chip exporter (Malaysia)

~13%

of global packaging & test

US\$130B

semiconductor exports, 2024

~5%

global sales from Penang

The dataset — UCI SECOM

Real wafer-fab process data; the defects are exactly what a data-management project must handle

1,567

production wafers

590

anonymized sensors

104

fails vs 1,463 passes

~41,951

missing values

~347

near-constant sensors

-1 / +1

pass / fail label

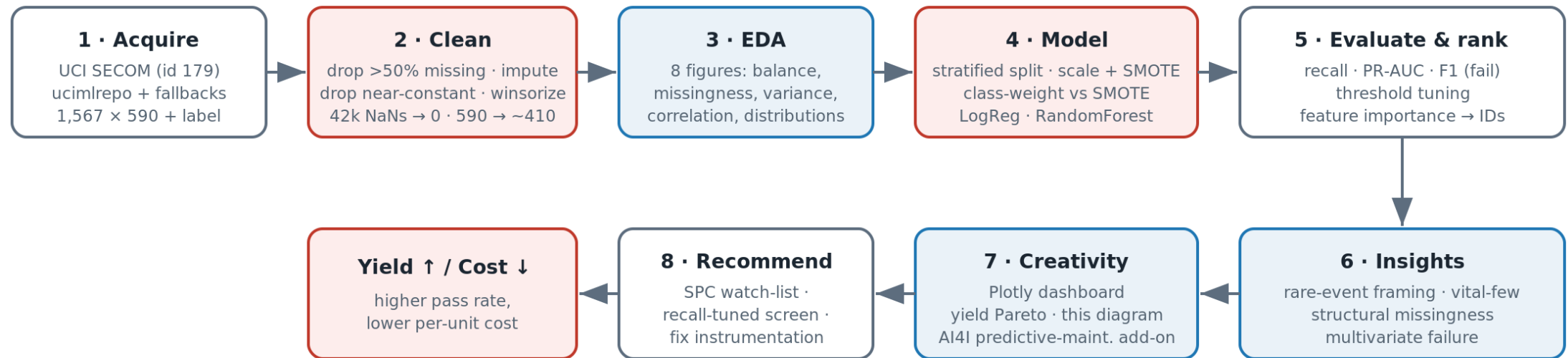
Honest limitations: features are anonymized (we report sensor IDs, not step names), only 104 failures cap achievable precision, and the AI4I add-on is synthetic and kept secondary.

The data-management pipeline

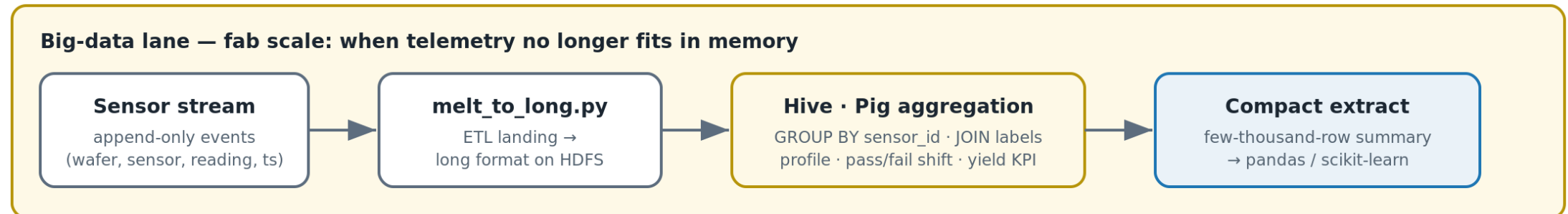
Industry problem to data to cleaning to modelling to insights — plus a fab-scale big-data lane

SECOM Yield Optimization — Data-Management Pipeline

Industry problem → data → cleaning → EDA → imbalance-aware modelling → insights → recommendations



Big-data lane — fab scale: when telemetry no longer fits in memory



Cleaning & preprocessing

Leakage-safe order, with a written justification and before/after audit for every step

1

Drop >50%-missing sensors

Can't impute more than half a column honestly.

2

Median imputation

Robust to the skew and spikes of sensor data.

3

Drop near-constant sensors

No discriminative power; only add noise.

4

Winsorize outliers

Clip, never delete — failures are precious.

5

Scale + SMOTE in-pipeline

Fit on the training fold only (no leakage).

Before → After

Sensors

590 → ~410

Missing cells

~41,951 → 0

Failure rows

104 → 104 (kept all)

Imbalance

handled via weights / SMOTE

Exploratory data analysis

Six-plus deliberate charts; red always means fail / risk



Accuracy is a trap

A 'predict pass' model scores ~93% and catches zero failures — we optimise recall / PR-AUC.

Signal is concentrated

A vital few sensor IDs carry most predictive power (yield Pareto).

Missingness is structural

It clusters in specific sensors offline for a period — a fixable data-quality gap.

Imbalance-aware modelling

Optimise for catching the rare failure — not headline accuracy

1

Validation

Stratified 75/25 hold-out keeps the 14:1 ratio in both splits.

2

Two remedies compared

Cost-sensitive class weights vs SMOTE, head-to-head.

3

Two models

Logistic Regression (interpretable) and Random Forest (interactions).

4

Honest metrics

Recall, precision, F1 and PR-AUC for the fail class — never accuracy.

5

Threshold tuning

Maximise fail-recall within a false-alarm budget.

6

Leads, not black box

Feature importance to ranked sensor IDs + a yield Pareto.

Insights — the 'why', not just the 'what'

Data-driven findings tied back to semiconductor context

Rare-event detection

With ~14:1 imbalance the task is catching a rare event; the whole method follows from that.

The vital few

Of 590 sensors, a small minority carry the signal — exactly SECOM's documented feature-selection use.

Failure is multivariate

Top sensors overlap heavily; no single red line works, so monitor a small set together.

Structural missingness

Gaps mean sensors were offline — an instrumentation fix, not random noise.

Recommendations for the fab team

Realistic, finding-supported, highest-leverage first

1

Stand up a 'vital-few' SPC watch-list

Tighten control on the top sensor IDs — the cheapest yield win.

2

Deploy a recall-tuned early-warning screen

Flag at-risk wafers for review; keep an engineer in the loop, don't auto-scrap.

3

Fix the offline-sensor instrumentation gaps

Treat structural missingness as a data-quality maintenance ticket.

4

Confirm leads with designed experiments

Take top sensors into a DOE before changing recipes — importance is associational.

5

Retrain on a rolling cadence

Handle drift; use active learning to grow the scarce failure set.

Big-data engineering — one job, four engines

Choosing the engine by data volume and infrastructure is the data-management skill on show

Engine	Reach for it when	In this project
pandas / scikit-learn	Fits in RAM; iterative analysis & ML	secom_yield_analysis.ipynb
DuckDB	Bigger than pandas, want fast SQL, zero infra	Pig + DuckDB notebook
Hive (via Spark SQL)	Partitioned TB-scale tables; BI access	Pig + Hive/Spark notebook
Apache Pig	Long procedural ETL dataflows	runs in local mode
Real Hadoop + Hive	The authentic warehouse stack	Hadoop + Hive notebook

Rule of thumb: prototype & model in pandas; push recurring full-history aggregation down to the cluster (Hive/Pig) and pull back a small, query-ready extract.

Conclusion

Stop optimising for accuracy and stop monitoring all 590 signals equally. Catch the rare failure, watch the vital few — and the data pipeline becomes a durable lever on yield and per-unit cost.

1

Honest, leakage-safe pipeline

cleaning → imbalance-aware modelling → recall-focused evaluation

2

Vital-few sensor IDs

ranked engineering leads + a yield Pareto

3

Four working engines

pandas · DuckDB · Hive/Spark · Pig + real Hadoop+Hive

4

GitHub-ready pack

README, /src, /docs, /notebooks, diagram, reproducible